Step 5 to Step 9: Build and Test the LangGraph Agent

Step 5 - Build the LangGraph agent

Now comes the interesting part! Create **agent.py**. This file calls the other two APIs, while Ollama decides which API to use.

```
import requests
from fastapi import FastAPI
from pydantic import BaseModel

from langchain_ollama import ChatOllama
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition

app = FastAPI()

# Tool 1: Call Addition API
@tool
def add_numbers(a: int, b: int):
    """Use this tool when the user wants to add two numbers."""
    r = requests.get("http://127.0.0.1:8001/add",
                     params={"a": a, "b": b})
    return r.json()

# Tool 2: Call Information API
@tool
def get_info(topic: str):
    """Get information about Python, FastAPI, or LangGraph."""
    r = requests.get("http://127.0.0.1:8002/info",
                     params={"topic": topic})
    return r.json()

# Ollama model
tools = [add_numbers, get_info]
model = ChatOllama(model="qwen2.5:7b", temperature=0)
llm = model.bind_tools(tools)

# LangGraph agent node
def agent(state: MessagesState):
    messages = [
        SystemMessage(content="Use the available tools when needed.")
    ] + state["messages"]
    return {"messages": [llm.invoke(messages)]}

# Build LangGraph
graph_builder = StateGraph(MessagesState)
graph_builder.add_node("agent", agent)
graph_builder.add_node("tools", ToolNode(tools))
```

```
graph_builder.add_edge(START, "agent")
graph_builder.add_conditional_edges("agent", tools_condition)
graph_builder.add_edge("tools", "agent")

graph = graph_builder.compile()

# Third API: Ask the agent
class Question(BaseModel):
    question: str

@app.post("/ask")
def ask(data: Question):
    result = graph.invoke({
        "messages": [HumanMessage(content=data.question)]
    })
    return {"answer": result["messages"][-1].content}
```

This uses LangGraph's standard agent loop: the model requests a tool, ToolNode executes it, and the model receives the result before producing its answer.

Step 6 - Start API 3

Open Terminal 3 in VS Code:

```
uvicorn agent:app --port 8003 --reload
```

Open the Swagger UI: http://127.0.0.1:8003/docs

You now have three FastAPI applications running on three different ports.

Application	Purpose	Address
API 1	Addition	http://127.0.0.1:8001
API 2	Information	http://127.0.0.1:8002
API 3	LangGraph agent	http://127.0.0.1:8003

Keep all three terminals running, along with the Ollama service.

Step 7 - Test your agent

In Swagger UI for API 3, find **POST /ask**, click Try it out, and enter one of these request bodies.

```
{ "question": "Add 10 and 20" }
{ "question": "Tell me about Python" }
{ "question": "What is LangGraph?" }
```

For the addition question, the expected answer is similar to:

```
{
  "answer": "The sum of 10 and 20 is 30."
}
```

The wording may differ because Ollama generates the final answer.

What have you actually learned?

- **FastAPI:** How to expose your own API endpoints.
- **requests:** How one API calls another API.
- **Ollama:** How to run the language model locally.
- **LangGraph:** How to control the model, tool, model workflow.
- **Tool calling:** How the model selects between the addition and information APIs.

One important detail: this demo assumes your Ollama model supports tool calling. If it responds without calling a tool, check the Ollama model version and its tool-calling support.

Your next step: Run API 1 and API 2 first and test both through Swagger. Then start API 3 and test the agent. This way, if something fails, you know which component to troubleshoot.